

```
In [1]: ## Load required modules
import zipfile
import os

zip_path = 'C:/Users/ADMIN/Desktop/Data Science/Datasets/Computer Vision/Diabeti

## Unzip the file
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall('Diabetic Retinopathy')

## Check directories
print("Root contents:", os.listdir('Diabetic Retinopathy'))

## Explore further
print("Inside Diabetic Retinopathy:", os.listdir('Diabetic Retinopathy/Diabetic
```

Root contents: ['Diabetic Retinopathy']

Inside Diabetic Retinopathy: ['test', 'train', 'valid']

Load and display images using matplotlib and PIL

```
In [2]: ## Load required modules
import matplotlib.pyplot as plt
from PIL import Image
import glob

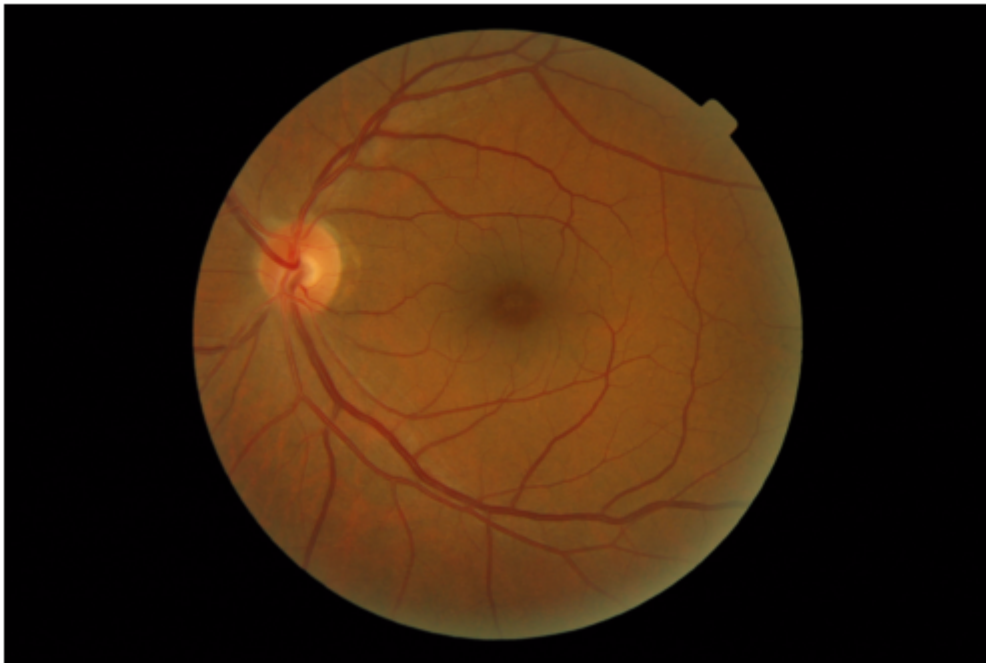
## Recursively find all image files in subdirectories
image_files = glob.glob('Diabetic Retinopathy/Diabetic Retinopathy/**/*.png', re
                  glob.glob('Diabetic Retinopathy/Diabetic Retinopathy/**/*.jpg', re
                  glob.glob('Diabetic Retinopathy/Diabetic Retinopathy/**/*.jpeg', r

## Check if any images were found
print("Total images found:", len(image_files))

## Load and display the first image
if image_files:
    image = Image.open(image_files[0])
    plt.imshow(image, cmap='gray') # use cmap='gray' for mammogram grayscale im
    plt.axis('off')
    plt.title("Sample Diabetic Retinopathy Image")
    plt.show()
else:
    print("⚠ No images found in the specified path.")
```

Total images found: 2254

Sample Diabetic Retinopathy Image



Augmentation

```
In [3]: ## Importing the image datagenerator
from tensorflow.keras.preprocessing.image import ImageDataGenerator

## Defining the generator with augmentation parameters
train_datagen = ImageDataGenerator(
    rescale=1./255, #normalize the pixels to range from 0-1
    shear_range=0.2, #sheared up to about 20% of its width and height
    zoom_range=0.2, #zoomed in and out up to 20%
    width_shift_range=0.2, #an image can shift up to 20% of its width
    height_shift_range=0.2, #an image can shift up to 20 % of its height
    horizontal_flip=True, # swap the image left and right
    fill_mode = 'nearest' # fill the empty pixels (after transformation ) wi
)
```

Loading the training images

```
In [4]: train_generator = train_datagen.flow_from_directory(
    'Diabetic Retinopathy/Diabetic Retinopathy/train',
    target_size = (256, 256), # resize all images to a fixed input size
    batch_size = 32, # process 32 images per time instead of processing imag
    class_mode = "binary",# for 0 and 1 classes
    color_mode = 'grayscale' # Load grayscale images
)

#Loading the testing split.
#We will only normalize the testing split since the model expects images of pixe
test_datagen = ImageDataGenerator(rescale=1./255) #parameter for normalization
```

Found 1577 images belonging to 1 classes.

LOADING THE TESTING SPLIT

```
In [5]: test_generator = test_datagen.flow_from_directory(
    'Diabetic Retinopathy/Diabetic Retinopathy/test', #path to the testing
```

```
target_size=(256, 256), #resizing the testing split
batch_size=32,
class_mode='binary',
color_mode = "grayscale")
```

Found 338 images belonging to 1 classes.

BUILDING THE MODEL ARCHITECTURE

```
In [6]: ## Load required modules
import tensorflow as tf
from tensorflow.keras import layers, models

## Mefining the model
model = models.Sequential()

## 1st Layer
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(256, 256, 1)

## Maxpooling
model.add(layers.MaxPooling2D((2, 2)))

## 2nd Layer
model.add(layers.Conv2D(64, (3, 3), activation='relu'))

## Maxpooling
model.add(layers.MaxPooling2D((2, 2)))

## 3rd Layer
model.add(layers.Conv2D(128, (3, 3), activation='relu'))

## Maxpooling
model.add(layers.MaxPooling2D((2, 2)))

## Flattening
model.add(layers.Flatten())

## Dense Layers (I only used two layers )
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(1, activation='sigmoid'))
```

C:\Users\ADMIN\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
In [7]: ## Compiling the model
model.compile(optimizer='adam', #optimizer to adjust the Learning rate as the mo
            loss='binary_crossentropy', # binary_crossentropy as our loss fun
            metrics=['accuracy'] # report accuracy during prediction
            )
## The model summary
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 254, 254, 32)	
max_pooling2d (MaxPooling2D)	(None, 127, 127, 32)	
conv2d_1 (Conv2D)	(None, 125, 125, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 62, 62, 64)	
conv2d_2 (Conv2D)	(None, 60, 60, 128)	
max_pooling2d_2 (MaxPooling2D)	(None, 30, 30, 128)	
flatten (Flatten)	(None, 115200)	
dense (Dense)	(None, 64)	
dense_1 (Dense)	(None, 1)	

Total params: 7,465,601 (28.48 MB)

Trainable params: 7,465,601 (28.48 MB)

Non-trainable params: 0 (0.00 B)

```
In [9]: ## Training the model
history = model.fit(train_generator, #training split
                    epochs = 2, #number of epochs
                    validation_data = test_generator, #testing split
                    verbose = 1 #print progress
                    )
```

Epoch 1/2

50/50 ————— 187s 4s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

Epoch 2/2

50/50 ————— 174s 3s/step - accuracy: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 - val_loss: 0.0000e+00

```
In [10]: ## Evaluating the model
loss, accuracy = model.evaluate(test_generator)
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)

loss, accuracy = model.evaluate(train_generator)
print("Train Loss:", loss)
print("Train Accuracy:", accuracy)
```

11/11 ————— 19s 2s/step - accuracy: 1.0000 - loss: 0.0000e+00

Test Loss: 0.0

Test Accuracy: 1.0

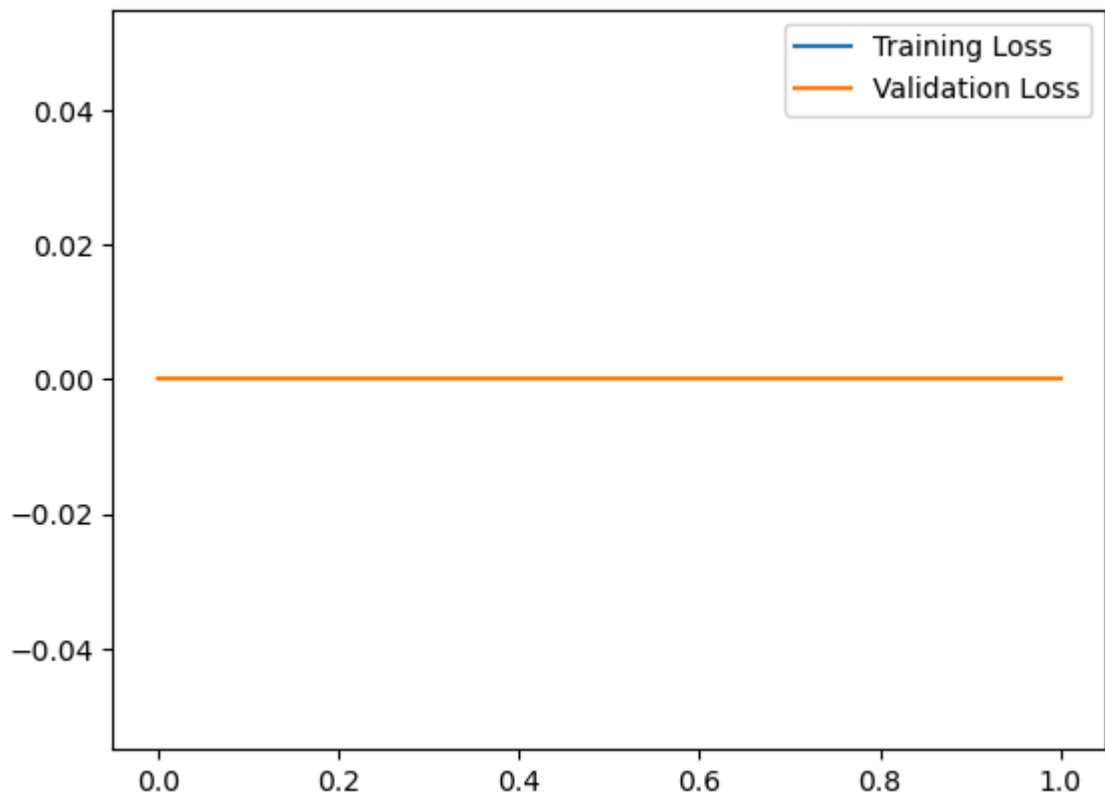
50/50 ————— 122s 2s/step - accuracy: 1.0000 - loss: 0.0000e+00

Train Loss: 0.0

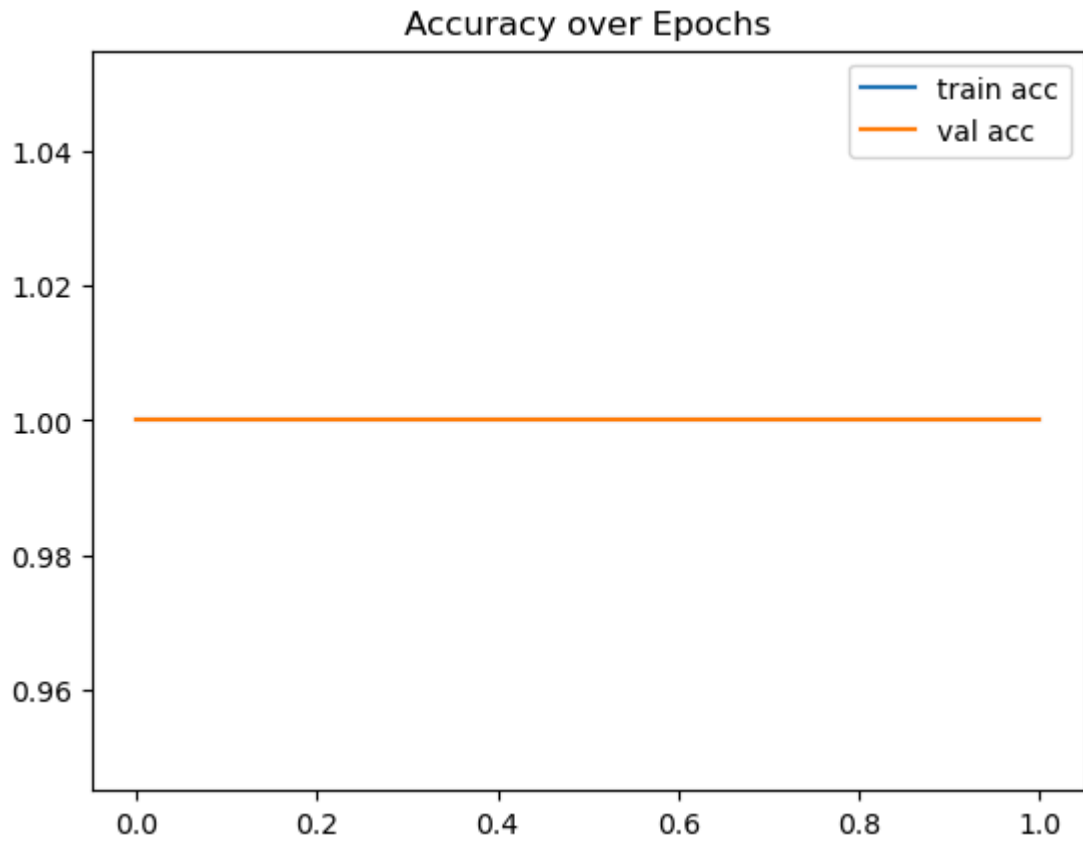
Train Accuracy: 1.0

```
In [11]: ## Overfitting check
import matplotlib.pyplot as plt
plt.plot(history.history["loss"], label = "Training Loss")
```

```
plt.plot(history.history["val_loss"], label = "Validation Loss")  
plt.legend()  
plt.show()
```



```
In [12]: ## Plot Training History  
plt.plot(history.history['accuracy'], label='train acc')  
plt.plot(history.history['val_accuracy'], label='val acc')  
plt.legend()  
plt.title("Accuracy over Epochs")  
plt.show()
```



```
In [13]: # Save the entire model in HDF5 format (best for Streamlit use)
model.save('Diabetes_Retinopathy_model.keras') #  Recommended modern format
```